

Performance profiles for the second-order radius rules on CUTEst

Abstract

PLACEHOLDER-ABSTRACT

1 What is measured

Three performance profiles, in this order of importance.

- P1** Among the second-order radius rules, on the second-order task. Only the radius rule differs across the lines, so this profile needs no further argument.
- P2** Second-order against first-order, scored on the *first-order* task. This is the overhead of carrying the curvature machinery when a first-order point is all that was wanted.
- P3** Second-order against first-order, scored on the *second-order* task. This is the payoff.

P2 and P3 together are how the two families compare. Neither alone is an answer. A report quoting only P2 says the second-order arm is expensive, a report quoting only P3 says it is reliable, and each has described half of one trade.

1.1 The rule that makes the comparison valid

Every run is scored afterwards, from the point it returned, against one criterion applied by the same code to every run. No profile in this report is drawn on the solver's own status.

The reason is that the two families stop on different tests. A first-order radius rule stops when the gradient is small. A second-order rule keeps going until it also has curvature. Profiling the two on their own stopping tests compares the cost of an easy task with the cost of a hard one, and the first-order rules win by construction.

Two tolerances are fixed once,

$$\varepsilon_g = 10^{-5}, \quad \varepsilon_H = 10^{-6}. \quad (1)$$

Both are the package's shared settings, ε_g the `tol` of `SOLVER_PARAMS` and ε_H the `tol_H` of `SECOND_ORDER_PARAMS`. The two criteria are

$$\text{first-order : } \|g\| \leq \varepsilon_g, \quad (2)$$

$$\text{second-order : } \|g\| \leq \varepsilon_g \text{ and } \lambda_{\min} \geq -\varepsilon_H. \quad (3)$$

Both are evaluated by `second_order_status` of the package, which is the same function the solver's own stopping test uses, called here on the returned point of every run whatever that run's status was.

2 Three things established before the runs

2.1 The subproblem solver

Part II asks the step for a fraction of the decrease available along the leftmost eigenvector. Assumption as: `Eigenpoint decrease` requires

$$m_k(0) - m_k(s_k) \geq \frac{1}{2}\kappa_{\text{fed}} |\lambda_k^-| \Delta_k^2 \quad \text{whenever } \lambda_k^- < 0, \quad (4)$$

and as: `max Cauchy/eigen decrease` asks for the better of that and the Cauchy decrease. Theorem `thm: Eigenpoint decrease` of Part II supplies (4) for the eigenpoint $s_k^E = \Delta_k u_k$ of `eqn: eigenpoint`, with u_k any direction satisfying `eqn: sufficient negative curvature direction`.

Three subproblem solvers of the package were examined against that condition.

ExactMS

Solves the trust-region subproblem exactly by the Moré–Sorensen characterisation, hard case included, so it dominates both the Cauchy point and the eigenpoint and satisfies (4) with $\kappa_{\text{fed}} = 1$. It needs a dense eigendecomposition and refuses above its `nmax`, which is 200.

EigenPoint(SteihaugCG())

Computes $\pm\Delta v_{\min}$ and returns whichever of that and the inner step decreases the model more, so it satisfies (4) with $\kappa_{\text{fed}} = \kappa_{\text{snc}}$, the quality of the eigenvector. That is 1 in the dense branch and Lanczos-limited above it.

SteihaugCG

Terminates on the first direction of negative curvature the conjugate-gradient recurrence reaches and runs to the boundary along it. The decrease is real and the direction is whichever the recurrence happened to find, so the guaranteed decrease carries no $|\lambda_k^-| \Delta_k^2$ term. It does not deliver the eigenpoint decrease.

The second-order arm therefore uses `ExactMS` at $n \leq 200$ and `EigenPoint(SteihaugCG())` above. The switch is on the problem, not on the column, so every line of a profile sees the same subproblem solver on a given problem and the per-problem normalisation is unaffected. The first-order arm uses the package’s default, `SteihaugCG`. Every caption names the pair.

2.2 The criticality measure

Part II defines the second-order criticality measure at `eqn:tau definition`,

$$\tau_k := \max\{\|g_k\|, -\lambda_k^-\} \geq 0, \quad (5)$$

with λ_k^- the leftmost eigenvalue of the *model* Hessian H_k . Part II states that τ_k is a property of the pair (x_k, H_k) and not of x_k alone, for that reason.

The package computes τ in `tau.criticality` as $\max\{\|g\|, -\min(\lambda_{\min}, 0)\}$. The inner clamp is a no-op inside the maximum, since $\|g\| \geq 0$. The two forms were compared on a grid of twenty pairs $(\|g\|, \lambda)$ covering $\lambda < 0$, $\lambda = 0$ and $\lambda > 0$, and the worst absolute difference is 0. So the package implements (5) exactly and no choice arises.

The open note at `app: measure` of the numerical study says that τ_k is the leftmost model Hessian eigenvalue in Part I and the whole second-order measure in Part II, and that Part III must pick one. **The premise is inaccurate and there is nothing to pick.** Part I defines no second-order criticality measure. Its criticality-anchored section carries a generic measure χ_k with $\chi_k = \|g_k\|$ as its stated example, at `eqn:criticality step`, and its concluding section says that second-order stationarity is outside the scope of its framework. The string `eigen` does

not occur anywhere in the Part I source, and its four occurrences of `lambda_` are the rates λ_1 and λ_2 of the adaptive update. The note should be resolved by recording that τ_k is (5) of Part II, which is what the package already implements.

2.3 Per-iteration costs, and the route taken

Post hoc scoring needs the cost a run had spent when it *first* met the criterion, not its total at the end. The package’s trace does not carry that. The trajectories it records are the radius, the gradient norm, the objective, the ratio, the step norm, the activity and acceptance flags, τ , the curvature and a further nine diagnostics. Not one of them is an evaluation count. The benchmark record adds three totals, read off the model after the solve, and no per-iteration counts either.

The fallback route was taken. Each configuration is run twice, with the tightened criterion as its *actual* stopping rule.

Pass g

halts at the first iterate with $\|g_k\| \leq \varepsilon_g$. Its cost is what P2 scores.

Pass h

halts at the first iterate that also has $\lambda_{\min} \geq -\varepsilon_H$. Its cost is what P1 and P3 score.

This is more runs and it is exact. Both passes keep their arm’s own `tol_H`, which in this package controls the curvature estimate as well as the stopping test. Switching it off in pass g would also switch off the cost that pass exists to measure, so pass g halts through a callback instead.

One arithmetic consequence of the package’s design bears on Section ?? . With `ExactHessian` at $n \leq 200$ the curvature estimate goes through `dense_hessian`, hence through `NLPModels.hess`, and is paid in *Hessian evaluations*. Above 200 it goes through Lanczos on `hessian_op` and is paid in Hessian-vector products. A profile on Hessian-vector products alone would therefore show the second-order arm as free on the small problems, which is the opposite of true. Both axes are reported.

3 The configurations

Six radius rules, in two groups of three.

Table 1: The roster. The second column is `is_criticality_anchored` and the third `asymptotic_regime`, both read from the package rather than assumed. ω_k is the measure the radius is anchored to in the second-order arm.

rule	reads a measure	regime	ω_k in the second-order arm
R-delta	no	<code>bounded_below</code>	not read
R-step	no	<code>step_summable</code>	not read
R-RTR	no	<code>step_summable</code>	not read
R-DFO	yes	<code>vanishing</code>	τ_k
R-grad	yes	<code>vanishing</code>	τ_k
R-gRTR	yes	<code>vanishing</code>	τ_k

The distinction in Table 1 governs how every caption in this report must be read. R-DFO, R-grad and R-gRTR read a criticality measure, so replacing $\|g_k\|$ by τ_k changes the radius rule itself. R-delta, R-step and R-RTR do not read one, so for them the second-order arm is *the same radius rule under a different stopping test and a different subproblem solver*. Both groups belong

in P1, and a caption that does not say which is which claims a distinction the figure does not have.

The two arms differ in exactly three things.

Table 2: The two arms. Everything not listed is identical, including the model Hessian, the acceptance and scaling thresholds, the contraction and expansion factors, the initial radius, the iteration cap and the two tolerances of (1).

	first-order arm	second-order arm
second-order stopping test	off	on, $\varepsilon_H = 10^{-6}$
ω_k , where read	$\ g_k\ $	τ_k
subproblem solver	SteihaugCG	ExactMS ($n \leq 200$), EigenPoint(SteihaugCG()) ($n > 200$)

The shared settings are those of `SOLVER_PARAMS`, unchanged: $\eta = \eta_1 = 0.1$, $\eta_2 = 0.9$, $\gamma_1 = 0.25$, $\gamma_2 = 0.5$, $\gamma_3 = 2$, $\Delta_0 = 1$, $\Delta_{\min} = 0$, and at most 10^4 iterations. R-DFO runs at $\zeta = 100$ and R-grad and R-gRTR at $\mu_0 = 1$ with no cap. The two retrospective rules run at their own $\tilde{\eta}_1 = 0.05$ and $\tilde{\eta}_2 = 0.9$. Per-rule tuning would measure tuning effort rather than algorithmic merit, which is why the settings are shared.

The model Hessian is `ExactHessian` in both arms. That choice is what makes the criterion (3) a statement about f . It was checked rather than assumed. On the problems tested, the model’s dense Hessian agrees with `NLPModels.hess` entry by entry, and the package’s $\lambda_{\min}$ agrees with a direct dense eigendecomposition, both to 0 in floating point.

3.1 LBFGSModel gets no line

`LBFGSModel` enforces $B \succ 0$, so $\lambda_{\min} > 0$ at every iteration, so $\tau_k = \|g_k\|$ identically and the second-order variant of a rule *is* its first-order variant. It therefore has no line of its own in P1.

This was verified on iterate sequences and not asserted. For each of the three criticality-anchored rules, on two problems, the wrapped and unwrapped rules were run over `LBFGSModel` and their iteration counts, gradient trajectories and radius trajectories compared. All six pairs are identical, entry by entry. There is no difference to report as a finding.

4 The problem set

CUTEst, unconstrained, with all variables free, $2 \leq n \leq 1000$ and no constraints. CUTEst version 1.4.0. The query returns **196** problems, of which **185** have $n \leq 200$ and **11** have $n > 200$. The realised list of names and dimensions is written beside the results, in `exp17_problems.txt`.

The fallback was disabled deliberately. `default_problems()` of the benchmark returns the analytic set when the CUTEst query comes back empty, so a run in which CUTEst failed to load produces a table that reads as a CUTEst benchmark and is not one. The experiment checks `HAS_CUTEST`, checks that the query returned something, and raises rather than falling back.

P3 runs on the 185 problems with $n \leq 200$ and no others. Above that dimension the package estimates $\lambda_{\min}$ by Lanczos, a Ritz value satisfies $\lambda_{\text{Ritz}} \geq \lambda_{\min}$, and an under-resolved run therefore *understates* the negative curvature and makes the second-order test optimistic. Scoring P3 on such an estimate would rank the runs by the machinery the profile exists to compare. On the 185 problems the estimate is a dense symmetric eigendecomposition of $\nabla^2 f(x)$ and is exact.

PLACEHOLDER-RESULTS